

Sprint Agent — Technical Reference Document

System: Business Analyst AI

Component: Sprint Planning Agent

Version: 1.0

Date: May 2026

Author: Business Analyst AI Platform

Table of Contents

1. [Overview](#)
 2. [Architecture Diagram](#)
 3. [Prerequisites & Trigger Conditions](#)
 4. [API Endpoint](#)
 5. [Internal Tool Pipeline](#)
 - 5.1 [StoryDecomposerTool](#)
 - 5.2 [StoryPointerTool](#)
 - 5.3 [MVPClassifierTool](#)
 - 5.4 [TeamSizerTool](#)
 - 5.5 [SprintAllocatorTool](#)
 6. [LLM Prompt & Structured Output](#)
 7. [Fallback Strategy](#)
 8. [Output Schema \(SprintPlan\)](#)
 9. [State Integration](#)
 10. [Configuration Constants](#)
 11. [Frontend UI Mapping](#)
-

1. Overview

The **Sprint Agent** is an LLM-powered planning module that takes a fully analysed and approved project brief and converts it into a production-ready sprint plan. It operates as a single-node **LangGraph subgraph** that combines a deterministic tool pipeline with an optional LLM enrichment call.

What it produces

Output	Description
Sprint Breakdown	2–10 two-week sprints with goals, features, and user stories
User Stories	One story per functional requirement (FR), with acceptance criteria and story points
Team Composition	Role-level breakdown: FE Dev, BE Dev, QA, DevOps, PM
Technology Stack	Component → Technology → Rationale mapping
Risk Register	Categorised risks with severity and mitigation plans
MVP Cutoff	Sprint number marking the Minimum Viable Product boundary

Design philosophy

- **Deterministic first, LLM-enriched second** — a fully working fallback pipeline runs first; the LLM call only improves the output, never breaks it.
- **Zero additional input required** — the agent reads everything it needs from `analyser_output` already in the project state.
- **Persisted to state snapshot** — the result is written back to both in-memory state and the SQLite snapshot immediately.

2. Architecture Diagram

Triggers the sprint planning agent for the given project.

Request

Part	Value
Method	POST
URL	/api/projects/{project_id}/sprint
Body	None required
Auth	None (internal service)

Response — 200 OK

```
{ "project_id": "abc-123-...", "sprint_plan": { "total_sprints": 6,
"sprint_duration_weeks": 2, "total_story_points": 84, "total_man_hours": 1680,
"mvp_cutoff_sprint": 4, "team_composition": [...], "technology_stack": [...],
"sprints": [...], "risk_register": [...], "generated_at":
"2026-05-14T10:23:41.000000+00:00" } }
```

5. Internal Tool Pipeline

The Sprint Agent runs **5 deterministic internal tools** in a fixed sequence before calling the LLM. These tools build a guaranteed-correct fallback plan that the LLM then enriches.

5.1 StoryDecomposerTool

Input: `list[FunctionalRequirement]` from `analyser_output`

Output: `list[dict]` — raw user story objects

Converts each Functional Requirement (FR) into a raw user story using the format:

```
"As a user, I want to {fr.description.lower()}"
```

Each story gets:

- `story_id`: RAW-001, RAW-002, ...
- `title`: First 80 characters of the FR description

- `description`: "As a user, I want to..." format
- `acceptance_criteria`: From `fr.acceptance_hints`, or ["Functional and tested end-to-end."]
- `moscow`: The FR's MoSCoW priority (`must_have`, `should_have`, `good_to_have`)
- `req_id`: The FR's `req_id` (e.g. FR-001)

5.2 StoryPointerTool

Input: Raw stories from StoryComposerTool

Output: Same stories with `story_points` field added

Assigns story points based on MoSCoW priority:

MoSCoW Priority	Story Points
<code>must_have</code>	5
<code>should_have</code>	3
<code>good_to_have</code>	2

5.3 MVPClassifierTool

Input: Pointed stories from StoryPointerTool

Output: Same stories with `is_mvp: bool` field added

Classifies each story as MVP or post-MVP:

MoSCoW Priority	<code>is_mvp</code>
<code>must_have</code>	<code>true</code>
<code>should_have</code>	<code>true</code>
<code>good_to_have</code>	<code>false</code>

Note: The `mvp_cutoff_sprint` parameter is passed but the current classification is purely MoSCoW-based.

5.4 TeamSizerTool

Input: `recommended_team` dict from `analyser_output`

Output: `list[TeamMember]` — role breakdown

Derives team composition from the analyser's recommendation. If the recommended team has fewer than 4 roles, or only contains generic roles (BA, Tech Lead, QA Engineer), it falls back to the **spec-default team**:

Role	Count	Hours/Sprint
FE Dev	2	80
BE Dev	2	80
QA	1	40
DevOps	1	40
PM	1	20

Total capacity per sprint: 440 man-hours

If roles are valid, the tool maps known roles to predefined hour capacities:

- FE Dev / BE Dev → 80 hrs/sprint
- QA / DevOps → 40 hrs/sprint
- PM → 20 hrs/sprint
- Unknown role → 60 hrs/sprint (default)

5.5 SprintAllocatorTool

Input: MVP-tagged stories + team composition

Output: `list[Sprint]` — complete sprint objects

Distributes stories across sprints using round-robin allocation:

- **MVP stories** (must/should have) → Sprints **1 to 4** (MVP_CUTOFF_SPRINT)
- **Post-MVP stories** (good to have) → Sprints **5 to 6**

Each sprint automatically gets an overhead story:

```
story_id: "S{n}-00" title: "Sprint {n} setup & grooming" role: "PM" points: 2 req_id: "INFRA"
```

Roles are assigned to stories using round-robin from `role_names` list built from team composition.

Man-hours per sprint = `sum(role.count × role.hours_per_sprint)` for all roles.

6. LLM Prompt & Structured Output

After the tool pipeline, the agent calls the LLM via `call_structured_json(prompt, fallback)`.

Prompt sent to LLM

```
You are a senior agile delivery consultant. Given the functional requirements and recommended team below, generate a complete sprint plan. Rules: - Decide the number of sprints based on scope: small projects (<=5 FRs) = 2-3 sprints medium (6-15 FRs) = 4-6 sprints large (>15 FRs) = 7-10 sprints - Each sprint is 2 weeks. Set mvp_cutoff_sprint to roughly 70% of total sprints (round down, min 1). - Derive team composition, technology stack, and risks from the actual requirements. Do NOT use generic placeholders. - Every functional requirement must map to at least one user story. FUNCTIONAL REQUIREMENTS: {json of FRs} RECOMMENDED TEAM: {json of recommended_team} Return ONLY a valid JSON object with the SprintPlan structure. Do NOT wrap in markdown code fences. Return raw JSON only.
```

LLM Model

Configured via `.env`:

```
GROQ_API_KEY=... DEFAULT_MODEL_PROVIDER=groq
DEFAULT_MODEL_NAME=llama-3.3-70b-versatile
```

Dynamic sprint count logic (LLM-driven)

Project Size	FRs	Sprints	MVP Cutoff
Small	≤ 5	2–3	1–2

Medium	6–15	4–6	3–4
Large	> 15	7–10	5–7

*The deterministic fallback always uses **6 sprints / MVP cutoff at Sprint 4**. The LLM adapts this to actual project scope.*

7. Fallback Strategy

The `call_structured_json` function automatically returns the fallback if:

- No LLM API key is configured
- The LLM returns malformed / non-JSON output
- The LLM call times out or returns an error

```
result = call_structured_json(prompt, fallback)
```

This means **the Sprint Agent never fails** — it always returns a sprint plan, even without an LLM connection. The LLM only makes the plan more intelligent and context-aware.

8. Output Schema (SprintPlan)

Full TypedDict definition from `app/shared/state_types.py`:

```
class SprintPlan(TypedDict):
    total_sprints: int # e.g. 6
    sprint_duration_weeks: int # always 2
    total_story_points: int # sum of all sprint points
    total_man_hours: int # sum of all sprint man_hours
    mvp_cutoff_sprint: int # sprint number where MVP is reached
    team_composition: list[TeamMember]
    technology_stack: list[TechStackItem]
    sprints: list[Sprint]
    risk_register: list[SprintRisk]
    generated_at: str # ISO 8601 UTC timestamp
    class TeamMember(TypedDict):
        role: str # e.g. "FE Dev"
        count: int # e.g. 2
        hours_per_sprint: int # e.g. 80
    class TechStackItem(TypedDict):
        component: str # e.g. "Frontend"
        technology: str # e.g. "React + TypeScript"
        rationale: str # Why this was chosen
    class Sprint(TypedDict):
        sprint_number: int
        sprint_name: str # e.g. "Sprint 1"
        goal: str # Sprint delivery goal
        features: list[str] # Feature names covered
        stories: list[UserStory]
        total_points: int
        man_hours: int
        is_mvp_cutoff: bool # True for the MVP boundary sprint
    class UserStory(TypedDict):
        story_id: str # e.g. "S1-01"
        title: str
        description: str # "As a user, I want to..."
        acceptance_criteria: list[str]
        story_points: int # 2, 3, or 5
        role: str # Assigned team role
        req_id: str # Source FR ID
    class SprintRisk(TypedDict):
        risk_id: str # e.g. "RISK-001"
        description: str
        category: str # "technical" | "business" | "delivery"
        severity: str # "high" |
```



```
"medium" | "low" mitigation: str sprint_impacted: int | None # Sprint number affected (or null)
```

9. State Integration

The sprint plan is stored in `GraphState`:

```
class GraphState(TypedDict): ... sprint_plan: SprintPlan | None ...
```

Persistence flow

1. `sprint_plan_node` returns `{"sprint_plan": result}`
2. `workflow.run_sprint_planning` calls `state.update(updates)`
3. A `sprint_plan_ready` streaming event is emitted
4. `_save(state)` persists the updated state to both in-memory dict and SQLite snapshot
5. The API handler (`generate_sprint_plan`) additionally calls `save_state_snapshot`

Retrieving after server restart

If the server restarts, the sprint plan is recovered from the SQLite snapshot:

```
state = _PROJECT_STATES.get(project_id) # in-memory first if state is None: state = load_state_snapshot(project_id) # SQLite fallback
```

10. Configuration Constants

Defined at the top of `app/agents/sprint/nodes/plan.py`:

```
_SPRINT_DURATION_WEEKS = 2 # Length of each sprint _TOTAL_SPRINTS = 6 # Default total sprints (fallback only) _MVP_CUTOFF_SPRINT = 4 # Default MVP cutoff sprint (fallback only)
```

*These constants only apply when the **LLM is unavailable** or returns an incomplete response. When the LLM is active, sprint count and MVP cutoff are dynamically computed based on the*

number of FRs.

11. Frontend UI Mapping

The `SprintPage.tsx` component maps the `SprintPlan` response to the following UI sections:

UI Section	Source Field	Description
Summary Cards	<code>total_sprints,</code> <code>sprint_duration_weeks,</code> <code>total_story_points,</code> <code>total_man_hours,</code> <code>mvp_cutoff_sprint</code>	5 top-level metric cards
Team Table	<code>team_composition</code>	Role, count, hours/sprint columns
Tech Stack Table	<code>technology_stack</code>	Component, technology, rationale columns (hidden if empty)
Sprint Boards	<code>sprints[]</code>	Collapsible board per sprint; MVP cutoff sprint highlighted in green
User Story Cards	<code>sprints[].stories[]</code>	Expandable cards with description + acceptance criteria
Risk Register	<code>risk_register[]</code>	Severity colour-coded (red/amber/green), category badges, mitigation text

Zustand store

Sprint state is managed in `useAppStore`:

```
sprintPlan: SprintPlan | null; applySprintPlan: (plan: SprintPlan) => void;
```

Document generated for Business Analyst AI — Sprint Agent v1.0